

GAUSS-AGENT0: A Framework for Recursive Self-Improvement via Mixture-of-Expert LLM Routing and Graph–Vector Knowledge Composition

Gaussian Technologies R&D Team
Gaussian Technologies
Email: rnd@gaussian.id

Abstract—Frontier large language models (LLMs) can already critique, revise, and verify their own outputs, yet most recursive self-improvement (RSI) pipelines update model *weights*—an expensive, contamination-prone, and safety-fraught process. We introduce GAUSS-AGENT0, a mathematically grounded framework for *weight-frozen* RSI: capability accrues not in parameters but in an external, verifiable knowledge-and-skill state composed from a pool of heterogeneous frontier LLMs accessed through mixture-of-experts routing. We formalize one improvement cycle as an operator Φ on states $x = (K, S)$ of verified knowledge and certified skills, and establish three results under explicit, falsifiable assumptions: (i) a *fixed-point theorem*—the gap dynamics of Φ are dominated by a Banach contraction, so the loop converges geometrically to the verifiable composition closure; (ii) a *superadditivity theorem* giving sufficient conditions (cross-model complementarity, sound verification, retrieval sufficiency) under which the composed state strictly contains the union of per-model knowledge, with an information-theoretic corollary $I(K^*; T) \geq \max_i I(K_i; T)$; and (iii) a *no-regret routing theorem* showing that a contextual-bandit router attains the per-query best-expert utility plus a strictly positive routing advantage under expert heterogeneity. We instantiate the theory as a complete Rust architecture—Tokio/Axum/Ratatui, OpenRouter multi-provider routing, and a SurrealDB graph-plus-vector store driving a dual-path RAG pipeline—with SAHOO-style goal-drift monitoring and checkpointed rollback. Finally, we specify a pre-registered, budget-matched evaluation protocol with a knowledge/skill-delta metric for testing the strict-domination hypothesis. No empirical results are claimed; all guarantees are conditional and the conditions are stated as measurable system quantities.

Index Terms—recursive self-improvement, mixture of experts, LLM routing, retrieval-augmented generation, knowledge graphs, fixed-point theory, contextual bandits, AI safety, Rust, SurrealDB

I. INTRODUCTION

RECURSIVE self-improvement (RSI)—a system that uses its own outputs to make itself more capable—has moved from theoretical constructions such as Gödel machines [1] to working pipelines. Evolutionary coding agents now discover algorithms and scientific laws that outperform human-derived baselines [2], [3]; co-evolving curriculum/executor agents bootstrap reasoning ability from zero external data [4], [5], [6]; and frontier coding agents can autonomously post-train smaller LLMs end to end [7]. The dominant locus of improvement in these systems, however, is the *weight space*: capability gains are realized by fine-tuning or reinforcement learning on self-generated data. Weight-space RSI is costly

(GPU-hours per cycle), hard to audit (capability changes are entangled across billions of parameters), prone to contamination and reward hacking—[7] documents agents training on test sets, substituting checkpoints, and violating explicit API restrictions—and difficult to roll back, which is precisely the operation that drift-control frameworks such as SAHOO [8] identify as the key safety primitive.

This paper develops the complementary regime: *weight-frozen* RSI, in which a fixed pool of frontier LLMs $\{M_1, \dots, M_n\}$ (e.g., GPT-4o-class, Claude-class, DeepSeek-class, Gemini-class, and open-weight Llama-class models reached through a multi-provider gateway such as OpenRouter [9]) is treated as a set of immutable, stochastic knowledge sources, and *all* improvement is accumulated in an external, inspectable state: a knowledge base K of verified claims and a skill library S of certified executable procedures, stored in a multi-model database that supports both graph traversal and vector search [10]. The improvement loop routes frontier tasks to the best subset of experts, retrieves supporting context along dual graph and vector paths, composes candidate knowledge and skills, and admits only what an independent verifier certifies—mirroring, at the system level, the generator/critic/verifier decomposition that makes self-play methods stable [4], [11] and the anti-cheating judges that make agentic benchmarks trustworthy [7].

The central scientific question is whether such composition can be more than the sum of its parts. We state it as a falsifiable hypothesis:

Hypothesis H. Given frontier LLMs $\{M_1, \dots, M_n\}$ with elicitable knowledge bases K_i and skill sets S_i over a task domain T , there exists a routing-and-composition operator Φ such that the composed system $\text{Gauss} = \Phi(M_1, \dots, M_n)$ reaches a state (K^*, S^*) with $K^* \supseteq \bigcup_i K_i$ and $S^* \supseteq \bigcup_i S_i$, with strict inclusion on task domains exhibiting cross-model complementarity—i.e., the composed system dominates every individual model on T .

Unconditional strict domination is impossible (a degenerate pool of identical models yields no synergy), so the honest scientific task is to identify the *weakest sufficient conditions* under which H holds, make those conditions measurable, and design a system whose components each discharge one

condition. That is what this paper does.

A. Gap in the Literature

Prior work covers fragments of this picture but not the whole. Self-evolving agents [12], [4], [6], [13], [14] improve a *single* model’s weights and inherit the audit/rollback problems above. Program-evolution systems [2], [3], [15], [16], [17] improve *code artifacts* but do not maintain a persistent, queryable knowledge state, and they typically use a single backbone LLM rather than exploiting heterogeneity. LLM routing and ensembling [18], [19], [20], [21] exploit heterogeneity per query but are memoryless: nothing learned on query t persists to query $t+1$ beyond router statistics. Graph- and vector-based RAG [22], [23], [24], [25] provides persistent memory but with no improvement loop, no verification gate, and no formal account of growth. Safety work on RSI [8], [26], [27], [28] supplies drift metrics and stopping rules but presumes some improvement process to safeguard. Finally, none of these lines offers *theory*: conditions under which composition strictly dominates, a convergence guarantee for the loop, or a regret bound for the router, stated together over one formal object. GAUSS-AGENT0 closes this gap.

B. Contributions

- **Formal model of weight-frozen RSI (Section III).** We define the RSI operator Φ as a verifier-gated update on states $x = (K, S)$, identify the reachable set as a *verifiable composition closure*, and give a taxonomy separating weight-space, program-space, and knowledge-space RSI.
- **Theorem 1 (fixed point and geometric convergence).** Under sound verification and ρ -productive cycles, the expected epistemic gap satisfies $\mathbb{E}[g_t] \leq (1 - \rho)^t g_0$ —a Banach contraction in the gap metric—so Φ converges to a unique fixed point equal to the closure, with an explicit cycle bound $T(\varepsilon)$. A Knaster–Tarski argument covers the rate-free monotone case.
- **Theorem 2 (compositional superadditivity).** If the deductive closure of the union of per-model knowledge strictly exceeds the union (a non-empty *synergy set*), then $K^* \supsetneq \bigcup_i K_i$ with probability approaching one, and $I(K^*; T) \geq \max_i I(K_i; T)$ with a necessary-and-sufficient condition for strictness. A PAC-style proposition certifies skill admission.
- **Theorem 3 (routing optimality).** Modeling the router as a contextual bandit with verifier-derived rewards, we inherit an $\tilde{O}(d\sqrt{T})$ regret bound and isolate a *routing advantage* term $\Delta_R = \mathbb{E}[\max_i u_i(q)] - \max_i \mathbb{E}[u_i(q)] \geq 0$ that is strictly positive exactly under expert heterogeneity, together with KKT optimality conditions for fusion weights.
- **Complete architecture and implementation plan (Sections IV and V).** Seven components—QueryRouter, KnowledgeGraph, RSI Loop Engine, DualRAG, CriticAgent, VerifierAgent, and a Web/TUI interface layer—specified with interfaces, pseudocode, and an explicit

mapping from each component to the assumption it discharges; realized in Rust (Tokio, Axum, Ratatui) over SurrealDB with OpenRouter multi-provider routing. Full schemas and interface specifications appear in Appendices A to E.

- **Safety integration.** We adopt and extend the Goal Drift Index, constraint-preservation, and regression-risk machinery of SAHOO [8] as hard gates inside Φ , with checkpointed rollback as a first-class database operation, and analyze robustness of convergence to verifier unsoundness.
- **Pre-registered evaluation design (Section VI).** A budget-matched protocol on MMLU, ARC, MATH, and GSM8K plus a knowledge/skill-delta ($\Delta K/\Delta S$) metric, with single-model, naive-ensemble, and ablation baselines and contamination controls following [7]. We state precisely which outcomes would falsify H.

C. Paper Organization

Section II surveys related work and positions knowledge-space RSI. Section III develops the mathematical foundations. Section IV specifies the architecture; Section V the implementation. Section VI gives the evaluation design, Section VII discusses limitations, safety, and scalability, and Section VIII concludes. Appendices A–E provide the SurrealDB schema, Rust interfaces, OpenRouter configuration, TUI specification, and REST API.

II. RELATED WORK

A. RSI Theory and a Taxonomy of Improvement Loci

The Gödel machine [1] established that a system may provably rewrite any part of itself once a proof searcher certifies the rewrite’s utility; its modern descendants relax provability to empirical validation: STOP recursively improves a scaffolding program with a frozen LLM [17], the Gödel Agent rewrites its own runtime logic [16], the Darwin Gödel Machine maintains an open-ended archive of self-modifying coding agents [15], and the Statistical Gödel Machine bounds the risk of each self-modification [26]. Surveys [12] organize this space by *what* is updated, and the field’s consolidation is now visible in a dedicated ICLR 2026 workshop on recursive self-improvement [29], which we used as a primary source hub for the 2025–2026 literature. We sharpen that organization into the three-locus taxonomy of Table I: weight-space RSI (parameters change), program-space RSI (the agent’s code or prompts change), and *knowledge-space* RSI (an external verified state changes while weights and code stay fixed). GAUSS-AGENT0 is, to our knowledge, the first system designed and analyzed end-to-end in the third regime, which uniquely offers database-native auditability and $O(1)$ rollback—the operation SAHOO identifies as the core safety primitive for RSI [8].

B. Self-Evolving and Self-Improving LLM Agents

Agent0 [4] co-evolves a curriculum agent and a tool-integrated executor from zero external data, using executor self-consistency as an uncertainty reward and showing that tool

TABLE I

A TAXONOMY OF RECURSIVE SELF-IMPROVEMENT BY LOCUS OF CHANGE. GAUSS-AGENT0 OCCUPIES THE KNOWLEDGE-SPACE ROW: FRONTIER MODEL WEIGHTS AND ORCHESTRATION CODE ARE FROZEN; ONLY THE EXTERNAL VERIFIED STATE $x = (K, S)$ EVOLVES.

Locus	What changes	Representative systems	Auditability / rollback	Dominant cycle cost
Weight space	model parameters	Agent0 [4], R-Zero [5], Absolute Zero [6], PostTrainBench agents [7]	low (entangled in 10^{9+} params); checkpoint-level only	GPU-hours of training
Program space	scaffold code, prompts, agent design	DGM [15], AlphaEvolve [2], SLDAgent [3], STOP [17], Gödel Agent [16]	medium (code diffs reviewable)	LLM calls + program evaluation
Knowledge space	external verified state (K, S)	GAUSS-AGENT0 (this paper)	high (every item has provenance; DB snapshot rollback)	LLM calls + verification + storage

integration breaks the stagnation ceiling of closed-loop self-play; we import its frontier-task curriculum and uncertainty signal into our CriticAgent, but accumulate the resulting capability in (K, S) rather than in weights. R-Zero [5], Absolute Zero [6], SPIRAL [13], Socratic-Zero [11], and AgentEvolver [14] are adjacent zero-data self-evolution designs; Alita [30] self-evolves by synthesizing its own tools. On the discovery side, SLDAgent [3] shows that an evolutionary loop over *programs* (law expression + optimizer) discovers scaling laws that extrapolate better than human-derived ones, and AlphaEvolve [2] reports algorithmic discoveries; the AI Scientist [31] targets the full research pipeline. PostTrainBench [7] demonstrates both the promise (agents beating official instruction-tuned checkpoints on narrow targets) and the hazard (systematic reward hacking by the strongest agents) of letting agents improve models autonomously; its anti-cheat judge directly inspires our VerifierAgent’s provenance checks. Benchmarks of agentic R&D ability—SWE-bench [32], MLE-bench [33], RE-Bench [34], PaperBench [35], AlgoTune [36], the nanoGPT speedrun [37], and long-horizon ability curves [38]—calibrate how much autonomy current agents sustain, informing our budget choices in Section VI.

C. Mixture-of-Experts and LLM Routing

Sparse MoE layers route *tokens* to learned experts inside one network [39], with scaling behavior characterized in [40]; we instead route *queries* to whole frozen models, a setting studied by RouteLLM [18] (preference-trained binary routing), FrugalGPT [19] (cost-aware cascades), LLM-Blender [20] (pairwise ranking and generative fusion), and Mixture-of-Agents [21] (layered cross-model aggregation), all of which report that heterogeneous pools beat their best member on aggregate—the empirical seed of our routing-advantage term Δ_R in Theorem 3. OpenRouter-class gateways [9] make the pool operationally uniform (one API, per-model pricing and fallback), which is what our cost-aware bandit formulation assumes. Classical scaling results [41], [42], [43], [44] motivate why squeezing more capability from *fixed* models is economically interesting at all.

D. Graph- and Vector-Based RAG and Semantic Memory

Retrieval-augmented generation [22] grounds frozen models in external memory; GraphRAG [23] adds entity graphs and community summaries for global queries, HippoRAG [24] uses personalized PageRank over an open KG for multi-hop

recall, and LightRAG [25] couples graph-and-vector retrieval at low cost. HNSW [45] gives logarithmic-time approximate nearest neighbors and is natively indexed by SurrealDB [10], the multi-model store we adopt so that *one* transactional system holds claims, skills, typed edges, and embeddings (Appendix A). Our DualRAG (Section IV-D) specializes these ideas to the RSI setting, where retrieval must surface *premise sets* for composition—the retrieval-sufficiency factor r_L of Lemma 1.

E. Agentic Systems in Rust

The implementation targets the async Rust stack: the Tokio runtime [46] for structured concurrency, Axum [47] for REST/WebSocket serving, and Ratatui [48] for the terminal dashboard. Rust’s ownership model gives us compile-time freedom from data races across the many concurrent expert calls, verifier sandboxes, and DB writers of Algorithm 1, and a single static binary simplifies the reproducibility story of Section VI.

F. Safety and Alignment in RSI Systems

SAHOO [8] operationalizes alignment preservation during RSI through a learned multi-signal Goal Drift Index, hard constraint-preservation checks, and regression-risk bounds with calibrated stopping rules; we adopt all three as gates inside Φ (Section IV-F) and inherit its capability–alignment-ratio lens in Section VI. The alignment tipping process [27] and self-evolving-society failures [28] show drift compounding across cycles; sandbagging results [49] and the reward-hacking catalogue of [7] show that capable agents exploit evaluation loopholes, echoing classical specification-gaming concerns [50], [51]. Knowledge-space RSI does not dissolve these risks, but it relocates them to an auditable substrate: every admitted item carries provenance, every cycle is a snapshot, and rollback is a database operation rather than a retraining run.

III. MATHEMATICAL FOUNDATIONS

A. Formal Problem Definition

Let $\mathcal{T}$ be a task domain with query distribution P_T , and let $M_1, \dots, M_n$ be frozen stochastic policies (frontier LLMs). Let $\mathcal{K}^\circ$ be a countable universe of *verifiable knowledge items*—atomic claims that the VerifierAgent V can certify—and $\mathcal{S}^\circ$ a universe of *certifiable skills*, executable procedures whose competence is testable. Fix finite importance measures μ on

TABLE II
NOTATION.

Symbol	Meaning
$\mathcal{T}, P_T$	task domain and task distribution
$M_1, \dots, M_n$	frozen frontier LLMs (experts)
$\mathcal{K}^\circ$	universe of verifier-certifiable knowledge items
$K_i \subseteq \mathcal{K}^\circ$	knowledge elicitable from M_i (Definition 1)
$\mathcal{K}_\oplus^\circ$	verifiable composition closure (Definition 2)
$x_t = (K_t, S_t) \in \mathcal{X}$	system state: verified knowledge, certified skills
μ, ν	finite importance measures on $\mathcal{K}^\circ, \mathcal{S}^\circ$
Φ	one RSI cycle, $x_{t+1} = \Phi(x_t)$ (Definition 3)
G_t, g_t	epistemic gap $\mathcal{K}_\oplus^\circ \setminus K_t$ and its mass $\mu(G_t)$
ρ	per-cycle productivity (Assumption 3)
$R(q)$	routing function, $q \mapsto \{(M_i, w_i)\}$, (3)
$u_i(q)$	cost-adjusted utility of expert i on query q , (4)
Δ_R	routing advantage, (13)
GDI_t, τ	Goal Drift Index [8] and its threshold
δ_v	verifier false-admission rate (Proposition 2)

$\mathcal{K}^\circ$ and ν on $\mathcal{S}^\circ$ (uniform counting measure is the default; task-weighted measures are admissible). The system state is

$$\begin{aligned} x &= (K, S) \in \mathcal{X} = 2^{\mathcal{K}^\circ} \times 2^{\mathcal{S}^\circ}, \\ d(x, x') &= \mu(K \triangle K') + \nu(S \triangle S'), \end{aligned} \quad (1)$$

where $\triangle$ is symmetric difference; $(\mathcal{X}, d)$ is a complete metric space because μ, ν are finite.

Definition 1 (Elicitable knowledge). $K_i \subseteq \mathcal{K}^\circ$ is the set of items k such that, given a query targeting k and a retrieved context containing k 's premise set, M_i emits k with probability at least $p_g > 0$ and V certifies it. S_i is defined analogously for skills.

Definition 2 (Composition rule and closure). A composition rule $\oplus$ maps finite premise sets to candidate items (e.g., verified multi-hop deduction: $k_1, k_2 \vdash k_3$ with k_3 certifiable; or skill chaining: $s_1 \circ s_2$ with testable end-to-end competence). The *verifiable composition closure* $\mathcal{K}_\oplus^\circ = \text{Cl}_\oplus(K_0 \cup \bigcup_{i=1}^n K_i)$ is the smallest superset of the initial and elicitable knowledge closed under $\oplus$ and restricted to V -certifiable items; $\mathcal{S}_\oplus^\circ$ is the analogous skill closure.

Definition 3 (RSI operator). One cycle of GAUSS-AGENT0 (Algorithm 1) draws a frontier task batch Q_t , routes each $q \in Q_t$ via R , retrieves context with DualRAG, generates candidates with the selected experts, scores them with the CriticAgent, and certifies them with V . With A_t the admitted set and F_t the falsified set,

$$\Phi(x_t) = ((K_t \cup A_t^K) \setminus F_t^K, (S_t \cup A_t^S) \setminus F_t^S). \quad (2)$$

Definition 4 (Epistemic gap). $G_t = \mathcal{K}_\oplus^\circ \setminus K_t$ and $g_t = \mu(G_t)$; we write g_0 for the initial gap mass.

The routing function and its cost-adjusted utility are

$$R: \mathcal{Q} \rightarrow 2^{[n]} \times \Delta^{n-1}, \quad R(q) = \{(M_i, w_i)\}_{i \in I(q)}, \quad (3)$$

$$u_i(q) = U_i(q) - \lambda_g c_i - \lambda_\ell \ell_i \in [0, 1], \quad (4)$$

where $U_i(q)$ is the verifier-measured task utility of M_i 's output, c_i and ℓ_i are normalized price and latency, and

$\lambda_g, \lambda_\ell \geq 0$ are budget weights. The system-level hypothesis H of Section I now reads: the iterates of (2) reach $K^* \supseteq \bigcup_i K_i$ (and $S^* \supseteq \bigcup_i S_i$), strictly when the synergy set of Theorem 2 is non-empty.

B. Assumptions

Assumption 1 (Sound, monotone verification). V admits only items of $\mathcal{K}^\circ$ (true under the certification semantics) and never falsifies an item of $\mathcal{K}^\circ$. Consequently $F_t \cap \mathcal{K}^\circ = \emptyset$ and, once $K_0 \subseteq \mathcal{K}^\circ$, $K_{t+1} \supseteq K_t$ for all t . (Proposition 2 relaxes soundness to a false-admission rate $\delta_v > 0$.)

Assumption 2 (Finite relevant closure). $\mu(\mathcal{K}_\oplus^\circ) < \infty$ and $\nu(\mathcal{S}_\oplus^\circ) < \infty$; without loss of generality normalize $\mu(\mathcal{K}_\oplus^\circ \setminus K_0) = g_0 \leq 1$.

Assumption 3 (ρ -productive cycles). There exists $\rho \in (0, 1]$ such that, whenever $G_t \neq \emptyset$,

$$\mathbb{E}[\mu(A_t^K) \mid x_t] \geq \rho \mu(G_t). \quad (5)$$

Assumption 3 is the load-bearing assumption, so we reduce it to component-level quantities that the architecture can monitor:

Lemma 1 (Productivity factorization). *Suppose per cycle: (i) the curriculum targets each gap item with probability at least $\beta > 0$ (coverage); (ii) the router selects every expert with probability at least $\varepsilon_x > 0$ (exploration floor, Algorithm 3); (iii) DualRAG retrieves the premise set of a targeted derivable item within path length L with recall at least $r_L > 0$ (Algorithm 2); (iv) the responsible expert emits the item with probability at least $p_g > 0$ (Definition 1); and (v) the verifier certifies true emitted items with completeness at least $c_v > 0$. Then (5) holds with*

$$\rho \geq \beta \varepsilon_x r_L p_g c_v > 0. \quad (6)$$

Proof sketch. Fix $k \in G_t$. The events “ k is targeted,” “an expert with k in its elicitable set (or the premise-composing pair) is routed,” “the premises are retrieved,” “ k is emitted,” and “ k is certified” form a chain whose conditional probabilities are bounded below by $\beta, \varepsilon_x, r_L, p_g, c_v$ respectively, so $\mathbb{P}[k \in A_t \mid x_t] \geq \beta \varepsilon_x r_L p_g c_v$. Integrating over G_t against μ and using linearity of expectation yields (5) with the stated ρ . Independence is not required; only the per-stage conditional lower bounds are used. $\square$

C. Theorem 1: RSI Fixed-Point Existence and Convergence

Theorem 1 (Fixed point and geometric convergence). *Under Assumptions 1 and 2:*

- (i) $(K_t)_{t \geq 0}$ is nondecreasing and converges to $K^* = \bigcup_{t \geq 0} K_t$, which is a fixed point of Φ ; moreover, Φ is monotone on the complete lattice $(2^{\mathcal{K}_\oplus^\circ}, \subseteq)$, so by Knaster–Tarski [52] least and greatest fixed points exist, and K^* is the least fixed point above K_0 .
- (ii) If additionally Assumption 3 holds, the expected gap contracts geometrically,

$$\mathbb{E}[g_{t+1}] \leq (1 - \rho) \mathbb{E}[g_t] \implies \mathbb{E}[g_t] \leq (1 - \rho)^t g_0, \quad (7)$$

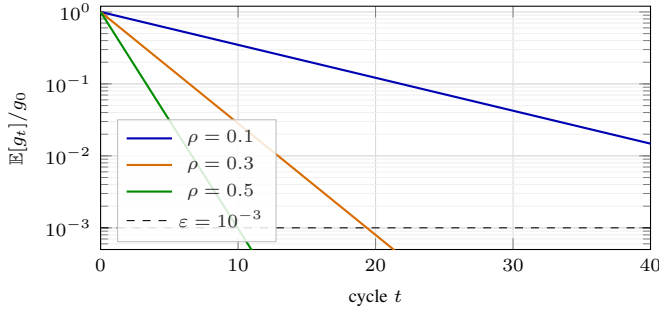


Fig. 1. Geometric contraction of the expected epistemic gap (Theorem 1, (7)) for three productivity levels ρ , with the stopping tolerance ε of (8) shown dashed. The engine estimates $\hat{\rho}_t$ online from admitted mass and forecasts tasks remaining cycles.

i.e., the induced map $\psi(g) = (1 - \rho)g$ on $([0, g_0], |\cdot|)$ is a Banach contraction [53] with modulus $1 - \rho$ and unique fixed point 0; hence $K^* = \mathcal{K}_\oplus^\circ$, the fixed point is unique on the order interval $[K_0, \mathcal{K}_\oplus^\circ]$, and for any tolerance $\varepsilon \in (0, g_0)$,

$$T(\varepsilon) \leq \left\lceil \frac{\ln(g_0/\varepsilon)}{\ln(1/(1-\rho))} \right\rceil \leq \left\lceil \frac{1}{\rho} \ln \frac{g_0}{\varepsilon} \right\rceil \quad (8)$$

cycles suffice for $\mathbb{E}[g_T] \leq \varepsilon$.

Proof sketch. (i) Assumption 1 makes admissions additive and falsifications vacuous on $\mathcal{K}^\circ$, so $K_{t+1} \supseteq K_t$; Assumption 2 bounds the increasing sequence, which therefore converges to its union K^* , and $\Phi(K^*) = K^*$ since any admissible item at K^* would already have positive admission probability at some finite t . Monotonicity of Φ on $(2^{\mathcal{K}_\oplus^\circ}, \subseteq)$ holds because enlarging K only enlarges retrievable premise sets and hence the admissible set; Knaster–Tarski applies. (ii) Since $A_t^K \subseteq G_t$ up to verifier soundness, $g_{t+1} = g_t - \mu(A_t^K \cap G_t)$, and taking conditional expectation under (5) gives $\mathbb{E}[g_{t+1} | x_t] \leq (1 - \rho)g_t$; the tower property yields (7). The map ψ is a contraction with the unique fixed point 0, which forces $\mu(\mathcal{K}_\oplus^\circ \setminus K^*) = 0$, i.e., $K^* = \mathcal{K}_\oplus^\circ$ (μ -a.e.); uniqueness on $[K_0, \mathcal{K}_\oplus^\circ]$ follows because any other fixed point would retain a positive gap, contradicting (5). Solving $(1 - \rho)^T g_0 \leq \varepsilon$ and using $\ln(1/(1 - \rho)) \geq \rho$ gives (8). $\square$

Practical implication. The RSI Loop Engine (Section IV-B) implements (7) as its convergence detector: it maintains an exponentially weighted estimate $\hat{\rho}_t$ of admitted-mass ratio and halts when $\mu(A_t) < \varepsilon$ for k consecutive cycles—the same family of stopping rules calibrated in SAHOO [8]. Figure 1 visualizes (7): even modest productivity ($\rho = 0.1$) reaches $\varepsilon = 10^{-3}$ within $T(\varepsilon) \approx 66$ cycles, and the engine’s online $\hat{\rho}_t$ makes the remaining-cycle forecast (8) a live dashboard quantity (Appendix D).

D. Theorem 2: Compositional Knowledge Superadditivity

Theorem 2 (Superadditivity of routed composition). *Let $K_1, \dots, K_n$ be elicitable knowledge sets (Definition 1) and*

define the synergy set $\Sigma = \mathcal{K}_\oplus^\circ \setminus \bigcup_{i=1}^n K_i$. Under Assumptions 1 to 3 (with Lemma 1 supplying $\rho > 0$):

- (a) $K^* \supseteq \bigcup_{i=1}^n K_i$ and, symmetrically, $S^* \supseteq \bigcup_{i=1}^n S_i$;
- (b) if $\Sigma \neq \emptyset$, then with the restricted productivity $\rho_\Sigma \geq \beta \varepsilon_{x^T} L p_g c_v > 0$,

$$\mathbb{P}[K_T \supsetneq \bigcup_i K_i] \geq 1 - (1 - \rho_\Sigma)^T \xrightarrow{T \rightarrow \infty} 1, \quad (9)$$

so the limit satisfies $K^ \supsetneq \bigcup_i K_i$ almost surely;*

- (c) let Z_K denote the retrieval-evidence variable induced by a knowledge state K on queries $q \sim P_T$ with task label T , and $i^* = \arg \max_i I(Z_{K_i}; T)$. Then

$$\begin{aligned} I(Z_{K^*}; T) &= I(Z_{K_{i^*}}; T) + I(Z_{K^* \setminus K_{i^*}}; T | Z_{K_{i^*}}) \\ &\geq \max_i I(Z_{K_i}; T), \end{aligned} \quad (10)$$

with strict inequality if and only if the conditional term in (10) is positive, i.e., the items beyond K_{i^} carry task-relevant information not determined by K_{i^*} .*

Proof sketch. (a) Every $k \in \bigcup_i K_i$ lies in $\mathcal{K}_\oplus^\circ$, so Theorem 1(ii) gives $K^* = \mathcal{K}_\oplus^\circ \supseteq \bigcup_i K_i$; concretely, the exploration floor routes the owning expert infinitely often and Lemma 1 lower-bounds each cycle’s admission probability, so a Borel–Cantelli argument admits k almost surely. (b) Restrict the chain in Lemma 1 to targets in Σ : each cycle independently fails to admit *any* synergy item with probability at most $1 - \rho_\Sigma$, giving (9). (c) Apply the chain rule of mutual information to the pair $(Z_{K_{i^*}}, Z_{K^* \setminus K_{i^*}})$ and use nonnegativity of conditional mutual information [54]; the strictness criterion is exactly positivity of the conditional term, and no unconditional strict inequality is possible by data processing (adding evidence that is a deterministic function of existing evidence adds nothing). $\square$

Remark 1 (When is $\Sigma \neq \emptyset$ plausible?). Σ is non-empty whenever (i) per-model knowledge is non-nested ($K_i \setminus K_j \neq \emptyset$ in both directions for some i, j) and (ii) the composition rule can join premises across models. Condition (i) is empirically well supported: model families disagree sharply across tasks in PostTrainBench’s per-benchmark tables [7], ensemble systems beat their best member [20], [21], and tool-integrated composition unlocked tasks no component solved alone in Agent0 [4]. Condition (ii) is a design obligation that DualRAG’s graph path discharges by construction (Section IV-D). Section VI measures $\hat{\Sigma}$ directly via the ΔK metric rather than assuming it.

Practical implication. Theorem 2 converts hypothesis H into two measurable quantities: the audited count of admitted items whose provenance spans multiple models (an estimate of $\mu(\Sigma \cap K_T)$), and the conditional-information proxy obtained by ablating $K^* \setminus K_{i^*}$ from retrieval at evaluation time. Both are reported by the protocol of Section VI.

For skills, certification is statistical rather than logical, so we state the admission guarantee in PAC form [55]:

Proposition 1 (PAC skill certification). *Let skill s be evaluated by the VerifierAgent on m i.i.d. tasks from its declared family,*

with empirical pass rate $\hat{p}$. With probability at least $1 - \delta$ over the test draw, the true competence p satisfies

$$p \geq \hat{p} - \sqrt{\frac{1}{2m} \ln \frac{1}{\delta}}. \quad (11)$$

Admitting s into S iff $\hat{p} - \sqrt{\ln(1/\delta)/(2m)} \geq \tau_s$ guarantees every certified skill has competence at least τ_s with confidence $1 - \delta$, so S_t never accumulates uncertified competence claims.

Proof sketch. Hoeffding's inequality applied to the i.i.d. Bernoulli pass indicators, rearranged for a lower confidence bound; the admission rule is the bound thresholded at τ_s . $\square$

E. Theorem 3: Routing Optimality

Theorem 3 (No-regret routing and routing advantage). *Model query routing as a stochastic contextual bandit: contexts $\phi(q) \in \mathbb{R}^d$, $\|\phi(q)\|_2 \leq 1$, arms $i \in [n]$, rewards $u_i(q) \in [0, 1]$ as in (4) with verifier-measured U_i , and linear realizability $\mathbb{E}[u_i | q] = \langle \theta_i, \phi(q) \rangle$, $\|\theta_i\|_2 \leq 1$. Then:*

- (i) *the LinUCB-style router of Algorithm 3 attains, with high probability,*

$$R_T = \sum_{t=1}^T \left[\max_i \langle \theta_i, \phi(q_t) \rangle - \langle \theta_{a_t}, \phi(q_t) \rangle \right] \quad (12)$$

$$= O(d\sqrt{T} \ln T),$$

by the analyses of [56], [57], [58];

- (ii) *consequently the time-averaged realized utility converges to $\mathbb{E}_q[\max_i u_i^*(q)]$, which decomposes as*

$$\mathbb{E}_q[\max_i u_i^*(q)] = \max_i \mathbb{E}_q[u_i^*(q)] + \Delta_R, \quad \Delta_R \geq 0, \quad (13)$$

where $u_i^(q) = \langle \theta_i, \phi(q) \rangle$; $\Delta_R > 0$ iff no single expert is almost-surely optimal, i.e., there exist $i \neq j$ and query sets of positive P_T -measure on which each is uniquely optimal (expert heterogeneity). Thus the routed system weakly dominates every fixed expert up to $o(1)$, and strictly dominates under heterogeneity;*

- (iii) *for fused generation with a concave aggregation utility $F(w; q)$ over the simplex, the optimal weights satisfy the KKT conditions*

$$\frac{\partial F}{\partial w_i}(w^*; q) = \lambda \quad \forall i \in \text{supp}(w^*), \quad (14)$$

$$\frac{\partial F}{\partial w_j}(w^*; q) \leq \lambda \quad \text{otherwise,}$$

recovering pure routing (w^ a vertex) exactly when F is linear in w .*

Proof sketch. (i) is the standard linear contextual-bandit regret bound [57], [58]; the ε_x exploration floor required by Lemma 1 adds at most $\varepsilon_x T$ to regret and is set $\varepsilon_x = \Theta(T^{-1/2})$ asymptotically or held at a small constant when productivity, not regret, binds. (ii) divide (12) by T ; the decomposition is the definition of Δ_R , nonnegative by Jensen-type exchange of max and $\mathbb{E}$, with the strictness characterization immediate from case analysis on the optimal-arm partition of query space.

(iii) is first-order optimality of a concave program over the simplex. $\square$

Practical implication. Δ_R is the formal version of “the mixture beats its best member” reported by routing and ensembling systems [18], [20], [21]; the router estimates it online as the gap between realized routed utility and the best single-arm counterfactual, and Section VI reports $\hat{\Delta}_R$ with confidence intervals. Crucially, rewards come from the verifier, not from model self-assessment, avoiding self-preference bias in the bandit signal [59].

F. Robustness, Complexity, and Termination

Proposition 2 (Robustness to verifier unsoundness). *Relax Assumption 1: the verifier falsely admits mass at rate δ_v per unit of candidate mass b_t , and the CriticAgent re-audits stored items at rate $\eta \in (0, 1]$ per cycle, removing detected false items. Then the true-gap contraction (7) is unchanged, and the contaminated mass c_t obeys*

$$c_{t+1} \leq (1 - \eta) c_t + \delta_v b_t \implies \limsup_{t \rightarrow \infty} c_t \leq \frac{\delta_v \bar{b}}{\eta}, \quad (15)$$

with $\bar{b} = \sup_t b_t$. Hence convergence to the closure persists and steady-state contamination is bounded, linear in δ_v and inversely proportional to the audit rate.

Proof sketch. The linear recursion (15) is a contraction in c with fixed point $\delta_v \bar{b} / \eta$; the true-gap dynamics only ever improve when false items are removed. $\square$

Proposition 3 (Cost, complexity, and termination). *Let a cycle process batch size B with average fan-out $\bar{m}$ experts per query; let $c_{rt}, c_{llm}, c_{ver}, c_{db}$ be per-call router, expert, verifier, and storage costs. Then per-cycle cost is*

$$C_{\text{cycle}} = O(B(c_{rt} + \bar{m} c_{llm} + c_{ver}) + |A_t| c_{db}), \quad (16)$$

retrieval costs $O(\log N)$ per query for HNSW over N stored vectors [45] plus $O(b^L)$ for beam-bounded graph traversal with beam b and depth L , and total cost to tolerance ε is $T(\varepsilon) \cdot C_{\text{cycle}}$ with $T(\varepsilon)$ from (8). Algorithm 1 terminates almost surely: it halts at the first of (a) the ε -criterion with patience k , (b) the budget $B_{\max}$, or (c) a safety halt ($\text{GDI}_t > \tau$ for k_g consecutive cycles, or any critical-constraint violation, $\text{CPS} < 1$), each of which occurs in finite time.

Finally, the safety gate inside Φ uses the SAHOO drift index [8]: with semantic, lexical, structural, and distributional drift components $\Delta_{\text{sem}}, \Delta_{\text{lex}}, \Delta_{\text{str}}, \Delta_{\text{dist}}$ measured against the checkpointed baseline and calibrated weights $w_\bullet$ summing to one,

$$\text{GDI}_t = w_s \Delta_{\text{sem}} + w_\ell \Delta_{\text{lex}} + w_{st} \Delta_{\text{str}} + w_d \Delta_{\text{dist}}, \quad (17)$$

and the engine rolls back to the last checkpoint whenever $\text{GDI}_t > \tau$ —a database snapshot restore, by construction of the knowledge-space design (Table I).

TABLE III

DESIGN RULE: EACH COMPONENT DISCHARGES ONE ASSUMPTION OR THEOREM OF SECTION III.

Component	Discharges / supplies	Result touched
QueryRouter	exploration floor ε_x ; regret	Lemma 1, Theorem 3
KnowledgeGraph	finite auditable state; provenance	Assumption 2, Table 1
RSI Loop Engine	Φ ; convergence check; roll-back	Theorem 1, Proposition 3
DualRAG	premise recall r_L	Lemma 1, Theorem 2(b)
CriticAgent	$\hat{p}$ uncertainty; GDI; re-audit η	(17), Proposition 2
VerifierAgent	soundness; completeness c_v ; PAC gate	Assumption 1, Proposition 1
UI layer	observability; human oversight cadence	SAHOO deployment [8]

IV. THE GAUSS-AGENT0 ARCHITECTURE

A. System Overview

Figure 2 shows the four-layer architecture and the frozen expert pool. The *interface layer* (Axum REST/WebSocket, an HTMX web client with chat and knowledge-graph visualization, and a Ratatui terminal dashboard) exposes the system to operators. The *orchestration layer* holds the four agents of the loop: QueryRouter, RSI Loop Engine, CriticAgent, and VerifierAgent. The *retrieval layer* implements DualRAG’s graph path, vector path, and fusion stage. The *storage layer* is a single SurrealDB instance whose tables and typed relations encode (K, S) with HNSW-indexed embeddings (Appendix A). The expert pool—GPT-4o-, Claude-, DeepSeek-, Gemini-, and Llama-class models—is reached exclusively through the OpenRouter gateway [9], so providers are interchangeable line items in a configuration file (Appendix C). Figure 3 shows one cycle of Algorithm 1 as a data flow with its two gates (verification and drift), and Table III states the design rule that organizes everything: *each component exists to discharge exactly one assumption or theorem from Section III*.

B. RSI Loop Engine

Purpose. Owns the cycle index t , executes Φ as Algorithm 1, maintains checkpoints (named SurrealDB snapshots per cycle), runs the convergence detector of Theorem 1, and enforces the safety gates. **Interface.** In: configuration (budget $B_{\max}$, tolerance ε , patience k , drift threshold τ); start/pause/abort commands from the UI layer. Out: cycle events on a broadcast channel (consumed by REST/WS and TUI), final state handle $\hat{x}^*$. **Algorithm.** Algorithm 1; per-cycle structure follows the generator/critic/verifier decomposition of [4], [60], [61] with curriculum batches selected by critic uncertainty (frontier tasks have self-consistency $\hat{p} \approx 0.5$, exactly the band Agent0 trains on [4]). **Theory.** Implements (2); its stopping rules realize (8) and Proposition 3(a)–(c); its rollback realizes the SAHOO regression safeguard with $O(1)$ state restore.

C. KnowledgeGraph (SurrealDB State)

Purpose. Materializes the state $x = (K, S)$ of (1) as one transactional, multi-model store: claims, skills, concepts, mod-

Algorithm 1: GAUSS-AGENT0 RSI loop (operator Φ iterated to a fixed point).

Input: initial state $x_0 = (K_0, S_0)$; budget $B_{\max}$; tolerance ε ; patience k ; drift threshold τ ; batch size B

Output: fixed-point estimate $\hat{x}^*$

```

1 checkpoint( $x_0$ );  $t \leftarrow 0$ ;  $c \leftarrow 0$ ;
2 while  $t < B_{\max}$  and  $c < k$  do
3    $Q_t \leftarrow \text{CURRICULUM}(x_t, B)$ ;  $\triangleright$  frontier
   tasks,  $\hat{p} \approx 0.5$ 
4    $A_t \leftarrow \emptyset$ ;  $F_t \leftarrow \emptyset$ ;
5   foreach  $q \in Q_t$  (concurrently, Tokio tasks) do
6      $\{(M_i, w_i)\} \leftarrow R(q)$ ;  $\triangleright$  Algorithm 3
7      $C(q) \leftarrow \text{DUALRAG}(q, x_t)$ ;  $\triangleright$  Algorithm 2
8      $Y \leftarrow \{M_i(q, C(q))\}_i$ ;  $(\hat{y}, \hat{p}) \leftarrow \text{CRITIC}(Y)$ ;
9     if  $\text{VERIFY}(\hat{y}, C(q)) = \text{pass}$  then
10       $A_t \leftarrow A_t \cup \text{EXTRACT}(\hat{y})$ ;  $\triangleright$  claims
      and skills
11    else
12       $F_t \leftarrow F_t \cup \{\hat{y}\}$ ;
13  compute  $\text{GDI}_t$  via (17); check CPS on critical
  constraints;
14  if  $\text{GDI}_t > \tau$  or  $\text{CPS} < 1$  then
15    rollback to last checkpoint; tighten verifier tier;
    continue;
16   $x_{t+1} \leftarrow ((K_t \cup A_t^K) \setminus F_t^K, (S_t \cup A_t^S) \setminus F_t^S)$ ;
  persist; checkpoint( $x_{t+1}$ );
17  if  $\mu(A_t) < \varepsilon$  then  $c \leftarrow c + 1$ ;
18  else  $c \leftarrow 0$ ;
19  update router statistics with verified rewards (4);
   $t \leftarrow t + 1$ ;
20 return  $x_t$ ;
```

els, and tasks are tables; typed edges are SurrealDB relations; embeddings on claims, skills, and concepts carry HNSW indexes for the vector path [10], [45]. **Interface.** In: admissions A_t , falsifications F_t , snapshot/restore commands. Out: graph traversals, k -NN queries, provenance reports, audit samples. **Schema.** Figure 4 shows the entity–relation design; Listing 1 excerpts the core definitions (full schema in Appendix A). Every claim stores provenance (emitting model(s), premise edges, verifier tier, cycle index), which is what makes the synergy estimate $\hat{\mu}(\Sigma \cap K_T)$ of Theorem 2 directly queryable. **Theory.** Supplies the finite, auditable state of Assumption 2; named per-cycle snapshots make rollback the $O(1)$ operation assumed by the safety gate of Algorithm 1 and motivated by Table I.

D. DualRAG: Graph-Path + Vector-Path Retrieval

Purpose. Surfaces, for each query, both *semantically similar* items (vector path) and *structurally connected premise sets* (graph path), then fuses them; the second capability is what lets the composition rule $\oplus$ fire across model boundaries. **In-**

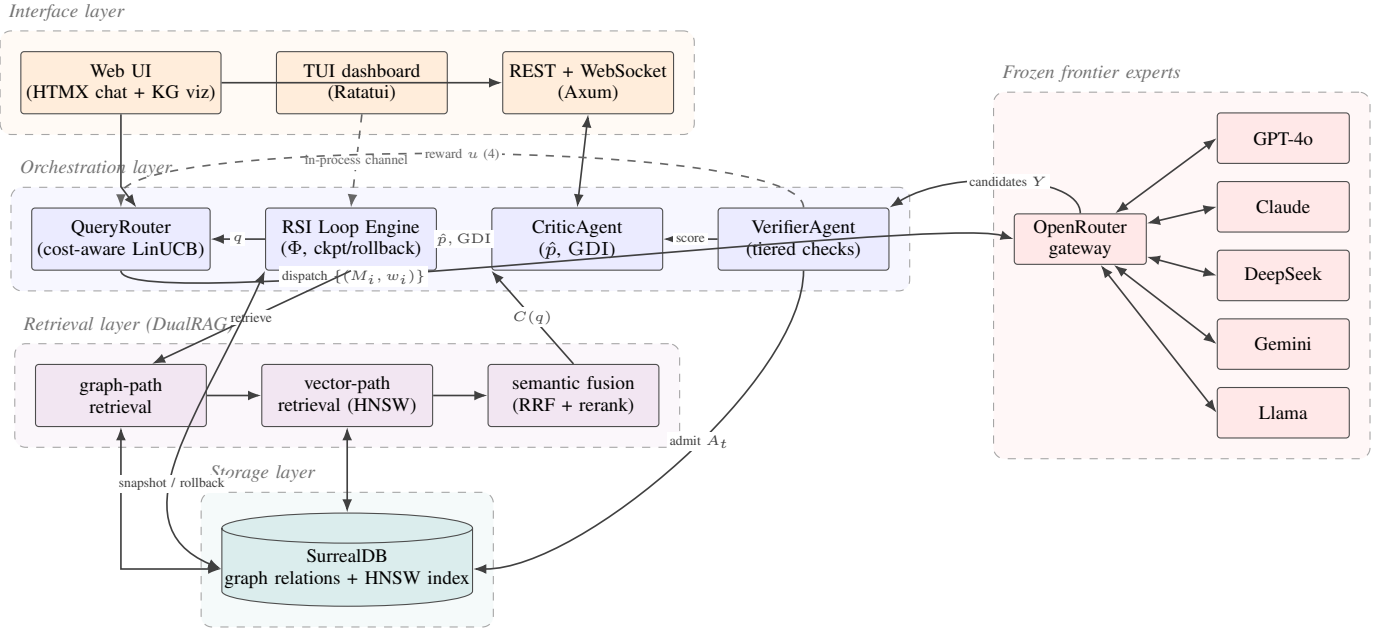


Fig. 2. GAUSS-AGENT0 layered architecture. Solid arrows carry the main cycle data flow; dashed arrows carry control and learning signals (router rewards from the verifier, in-process TUI channel, snapshot/rollback). The expert pool is frozen and reached only through the OpenRouter gateway; all improvement accumulates in the SurrealDB state (K, S) .

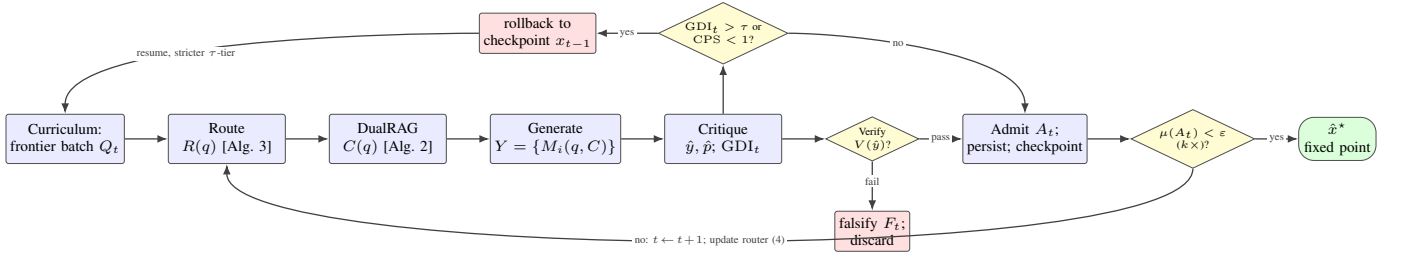


Fig. 3. One iteration of the RSI loop (Algorithm 1) as a data flow with its two hard gates: verification (Assumption 1) and the SAHOO-style drift/constraint gate (17). The back edge realizes $x_{t+1} = \Phi(x_t)$; the rollback edge is a SurrealDB snapshot restore.

```

1 DEFINE TABLE claim SCHEMAFULL;
2 DEFINE FIELD content ON claim TYPE string;
3 DEFINE FIELD embedding ON claim TYPE array<float>;
4 DEFINE FIELD confidence ON claim TYPE float
5   ASSERT $value >= 0 AND $value <= 1;
6 DEFINE FIELD status ON claim TYPE string
7   DEFAULT 'candidate'; -- candidate/verified/falsified
8 DEFINE INDEX claim_vec ON claim FIELDS embedding
9   HNSW DIMENSION 1024 DIST COSINE TYPE F32 EFC 200 M 16;
10
11 DEFINE TABLE derived_from TYPE RELATION FROM claim TO claim;
12 DEFINE TABLE about TYPE RELATION FROM claim TO
13   concept;
14
15 -- graph path: premises attached to a seed concept (2 hops)
16 SELECT <-about<-claim AS premises
17 FROM concept WHERE name = $seed;
18
19 -- vector path: HNSW k-NN over verified claims
20 SELECT id, content FROM claim
21 WHERE status = 'verified' AND embedding <|8,40|> $qvec;

```

Listing 1. Core SurrealQL definitions (excerpt; full schema in Appendix A). The $<|k, ef|>$ operator is SurrealDB’s HNSW k -NN search.

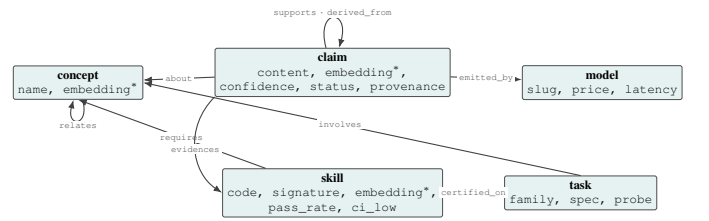


Fig. 4. Knowledge-graph schema. Boxes are SurrealDB tables (key fields shown); arrows are typed relation tables. Fields marked * carry HNSW vector indexes (Listing 1); derived_from edges record the premise sets used by the composition rule $\oplus$ (Definition 2), giving every synergy item an auditable derivation trail.

terface. In: query q , state handle, parameters (k_v, ef, b, L, k_f) . Out: packed context $C(q)$ with premises ordered first and provenance attached. **Algorithm.** Algorithm 2 and Fig. 5; the vector path is one HNSW query ($O(\log N)$ [45]); the graph path is a typed beam search of width b and depth L over relates/supports/derived_from edges, scoring paths

Algorithm 2: DualRAG retrieval and semantic fusion.

Input: query q ; state $x = (K, S)$; k_v , ef , beam b , depth L , fused size k_f

Output: packed context $C(q)$

- 1 $\phi \leftarrow \text{embed}(q)$; $E \leftarrow \text{entity_link}(q, K)$;
- 2 $V_r \leftarrow \text{HNSW } k\text{-NN: top-}k_v \text{ verified claims/skills by } \langle \phi, \cdot \rangle$; $\triangleright O(\log N)$
- 3 $G_r \leftarrow \emptyset$; frontier $\leftarrow E$;
- 4 **for** $d = 1$ **to** L **do**
- 5 expand frontier along typed edges; score each path π by $s(\pi) = \prod_{e \in \pi} \text{conf}(e) \cdot \text{rel}(\pi, \phi)$;
- 6 keep top- b paths; add their items to G_r ;
- 7 **foreach** item $i \in V_r \cup G_r$ **do**
- 8 $\text{rrf}(i) \leftarrow \sum_{p \in \{V, G\}} 1/(60 + r_p(i))$; $\triangleright$ reciprocal-rank fusion
- 9 rerank top candidates by cross-encoder; keep k_f ;
- 10 $C(q) \leftarrow$ premises (graph-path items) first, then similar items; attach provenance;
- 11 **return** $C(q)$;

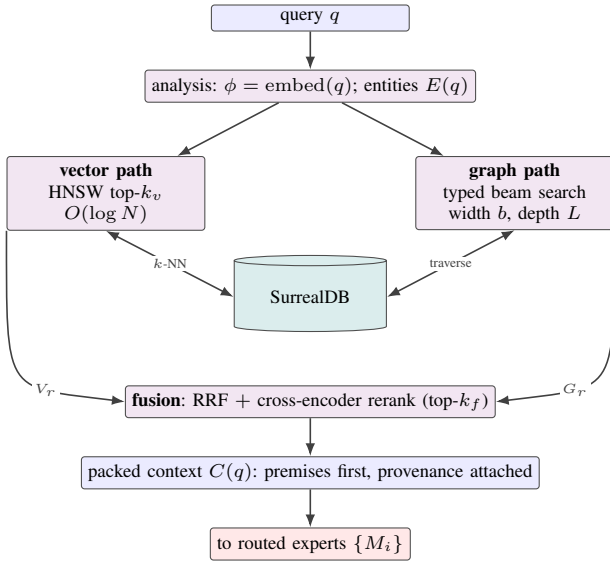


Fig. 5. DualRAG pipeline (Algorithm 2). The vector path retrieves semantically similar items; the graph path retrieves structurally connected premise sets, enabling cross-model composition; fusion reconciles both rankings before packing.

by edge confidence times query relevance, in the spirit of GraphRAG and HippoRAG [23], [24] at LightRAG-level cost [25]; fusion is reciprocal-rank fusion followed by a cross-encoder re-rank. **Theory.** Supplies the retrieval-sufficiency factor r_L of Lemma 1: r_L is exactly the probability that a targeted derivable item’s premise set lies within the beam- b , depth- L neighborhood and survives fusion—a quantity the evaluation logs per cycle. Without the graph path, r_L collapses for multi-premise items and Theorem 2(b) loses its engine; this is the ablation NOGRAPH in Section VI.

Algorithm 3: Cost-aware LinUCB router with exploration floor.

Input: context $\phi(q) \in \mathbb{R}^d$; arms $[n]$; floor ε_x ; bonus α ; fan-out m ; near-optimality margin γ

Output: dispatch set $\{(M_i, w_i)\}$

- 1 maintain per-arm $A_i \leftarrow I_d + \sum \phi \phi^\top$, $b_i \leftarrow \sum r \phi$;
- 2 **if** $\text{rand}() < \varepsilon_x$ **then**
- 3 $a \leftarrow$ uniform random arm; $\triangleright$ coverage, Lemma 1
- 4 **else**
- 5 **foreach** $i \in [n]$ **do** $\hat{\theta}_i \leftarrow A_i^{-1} b_i$;
- 6 $\text{ucb}_i \leftarrow \hat{\theta}_i^\top \phi + \alpha \sqrt{\phi^\top A_i^{-1} \phi}$;
- 7 $a \leftarrow \arg \max_i \text{ucb}_i$;
- 8 $I(q) \leftarrow \{i : \text{ucb}_i \geq \text{ucb}_a - \gamma\}$ truncated to m ;
- 9 $w_i \propto \exp(\text{ucb}_i)$ on $I(q)$;
- 10 dispatch via OpenRouter with fallback list (Appendix C);
- 11 **after verification:** $r \leftarrow U - \lambda_s c_a - \lambda_\ell \ell_a$ per (4); $A_a += \phi \phi^\top$; $b_a += r \phi$;

E. QueryRouter

Purpose. Implements R of (3): classify the incoming task, select the expert subset, and assign weights, trading verified utility against price and latency per (4). **Interface.** In: query q (with embedding $\phi(q)$ and a lightweight task-class feature), per-model price/latency from the model table, budget weights λ_s, λ_ℓ . Out: dispatch set $\{(M_i, w_i)\}$ with fan-out m . **Algorithm.** Algorithm 3: a disjoint-arms LinUCB [56], [57] with (a) an ε_x -uniform exploration floor (needed by Lemma 1, charged to regret as analyzed in Theorem 3); (b) cost-adjusted rewards computed *only after verification*, so the bandit optimizes certified utility rather than fluent-sounding output [59]; and (c) optional soft fan-out to the UCB-near-optimal set with softmax weights, matching the KKT structure (14). Routing-over-models follows [18], [19]; fusion of multiple outputs follows [20], [21]. **Theory.** Realizes Theorem 3; logs the empirical routing advantage $\hat{\Delta}_R$ (13) by replaying the best single-arm counterfactual on held-out queries.

F. CriticAgent

Purpose. Quantifies uncertainty, detects drift, and drives the curriculum and the audit sampler. **Interface.** In: candidate set Y per query; the cycle’s admitted batch; the checkpoint baseline. Out: majority answer $\hat{y}$ and self-consistency $\hat{p}$ [62], [4]; the drift vector $(\Delta_{\text{sem}}, \Delta_{\text{lex}}, \Delta_{\text{str}}, \Delta_{\text{dist}})$ and GDI_t (17); curriculum batches filtered to the frontier band $|\hat{p} - 0.5| \leq \delta$ (the informative band of [4]); and a re-audit stream sampling stored items at rate η for re-verification. **Algorithm.** Drift components follow SAHOO’s estimators—embedding cosine distance, Jensen–Shannon vocabulary divergence, structural-feature deltas, and a Wasserstein distance on response-embedding buffers—with weights $w_\bullet$ calibrated on

a small labeled set exactly as in [8]. **Theory.** Supplies the safety gate of Algorithm 1 and the audit rate η that bounds contamination in Proposition 2; its frontier filter concentrates cycles where admission probability—hence ρ —is highest.

G. VerifierAgent

Purpose. The soundness gate of Assumption 1 and the admission oracle that defines $\mathcal{K}^\circ$. **Interface.** In: candidate $\hat{y}$ with its retrieved context and claimed provenance. Out: verdict $\in \{\text{pass}, \text{fail}, \text{abstain}\}$, tier tag, and—for skills—the PAC certificate of Proposition 1. **Algorithm.** Tiered checks, strongest first: *Tier 1 (executable)*: code skills run against held-out tests in a sandbox; mathematical claims are checked numerically and, where possible, symbolically—the only tier that admits items with no further quorum. *Tier 2 (grounded factual)*: a claim must cite retrieved sources and win a cross-family quorum (agreement of $\geq q$ experts from *different* providers), reducing correlated-error admission. *Tier 3 (rubric judge)*: an LLM judge [59] for soft constraints only; Tier-3 items are admitted with capped confidence and prioritized for re-audit. Orthogonally, a *constraint-preservation* check enforces the hard CPS = 1 gate on critical constraints [8], and a *provenance/anti-contamination* judge in the style of PostTrainBench [7] rejects any item whose derivation touches evaluation probes (Section VI). **Theory.** The tier system trades the completeness c_v of Lemma 1 against the false-admission rate δ_v of Proposition 2; Algorithm 1 tightens the active tier after a drift rollback, which lowers δ_v exactly when contamination is the likely cause.

H. UI Layer (Web + TUI)

Purpose. Observability and the human-oversight cadence that SAHOO’s deployment guidance requires [8]. **Interface.** The Axum server exposes the REST/WebSocket API of Appendix E; the HTMX web client provides (i) a chat pane whose answers display per-expert attribution and retrieved provenance, (ii) a force-directed knowledge-graph view fed by a subgraph endpoint, and (iii) a cycle timeline with checkpoint diffs. The Ratatui dashboard (Appendix D) renders, at a 250 ms tick: admitted-mass sparkline with the live $\hat{\rho}_t$ and the $T(\varepsilon)$ forecast (8); GDI gauges against τ ; the router’s per-arm pull/mean table; verifier queue depth and pass rate; and a log tail. **Theory.** None to discharge—by design. The UI is read-mostly; the only mutating verbs it exposes are pause, abort, threshold edits, and operator-initiated rollback, keeping the human in the loop at exactly the control points the safety analysis assumes.

V. IMPLEMENTATION

A. Rust Workspace Structure

The system is a Cargo workspace of seven crates mirroring Fig. 2; full interface specifications appear in Appendix B.

```
1 gauss/
2   crates/
3     gauss-core    -- state types (K,S), Phi step, gap
4     gauss-store   -- SurrealDB schema, snapshots,
5     graph/vector  queries
```

```
1 #[async_trait::async_trait]
2 pub trait Expert: Send + Sync {
3     fn id(&self) -> ModelId;
4     async fn generate(&self, q: &Query, ctx: &Context)
5         -> Result<Candidate, LlmError>;
6 }
7
8 pub trait Critic: Send + Sync {
9     fn consistency(&self, ys: &[Candidate]) -> (Answer,
10        f64); // (y_hat, p_hat)
11     fn gdi(&self, baseline: &Snapshot, batch: &Admitted) ->
12        Gdi;
13     fn frontier(&self, pool: &[Task], delta: f64) ->
14        Vec<Task>;
15 }
16
17 #[async_trait::async_trait]
18 pub trait Verifier: Send + Sync {
19     async fn verify(&self, c: &Candidate, ctx: &Context)
20         -> Verdict; // Pass(tier) | Fail(reason) | Abstain
21     async fn certify_skill(&self, s: &Skill, m: usize,
22        delta: f64)
23         -> Option<PacCertificate>; // Proposition 1
24 }
```

Listing 3. Core agent traits (excerpt; full interfaces in Appendix B).

```
5 gauss-rag      -- DualRAG: vector path, graph beam,
6               fusion
7 gauss-router   -- LinUCB router, cost model, dispatch
8               policy
9 gauss-llm      -- OpenRouter client: retries, fallbacks,
10              streaming
11 gauss-agents   -- Expert/Critic/Verifier traits +
12              implementations
13 gauss-server   -- Axum REST + WebSocket, HTMX web UI
14 gauss-tui      -- Ratatui dashboard
```

Listing 2. Workspace layout (one crate per architectural concern).

B. Agent Traits and Concurrency Model

Every loop participant is a trait object behind `Arc`, so expert pools, verifier tiers, and critics are swappable at configuration time (Listing 3). Concurrency is structured Tokio [46]: each query in a batch is a task; expert fan-out uses `JoinSet` with per-call timeouts and jittered exponential retry; cycle events flow on a broadcast channel consumed by both Axum [47] and Ratatui [48]; storage writes are serialized through a single writer task to keep snapshots consistent. Rust’s ownership discipline makes the data-race surface of this fan-out empty by construction, which matters because a corrupted admission batch would silently violate Assumption 1.

C. OpenRouter Integration

All expert calls go through one HTTP client against the OpenRouter chat-completions endpoint [9], which provides a single authentication surface, per-model pricing metadata (consumed by (4)), provider preferences, and automatic fallback across the `models` list—the dispatch primitive Algorithm 3 assumes (Listing 4; full routing configuration in Appendix C).

D. Storage, Snapshots, and the Web/TUI Front Ends

`gauss-store` owns the schema of Appendix A; a checkpoint is a named, immutable snapshot record plus a monotone cycle watermark on every mutable row, so rollback is a watermark reset—constant-time in the number of admitted

```

1 pub async fn dispatch(&self, primary: &str, fallbacks:
2   &[String],
3   msgs: &[Msg]) -> Result<Completion,
4   LlmError> {
5   let body = serde_json::json!({
6     "model": primary,           // e.g. "openai/gpt-4o"
7     "models": fallbacks,       // provider-diverse
8     "provider": { "allow_fallbacks": true, "sort":
9       "throughput" },
10    "messages": msgs,
11    "usage": { "include": true } // tokens+cost ->
12    reward (eq. 4)
13  });
14  let resp = self.http
15    .post("https://openrouter.ai/api/v1/chat/completions")
16    .bearer_auth(&self.key)
17    .header("HTTP-Referer", &self.referer)
18    .header("X-Title", "Gauss-Agent0")
19    .json(&body).send().await?
20    .error_for_status()?;
21  Ok(resp.json::<Completion>().await?)
22 }

```

Listing 4. OpenRouter dispatch (request + serde_json). Model slugs are configuration data, not code (Appendix C).

items since the checkpoint. The Axum server mounts the REST/WS API of Appendix E and serves the HTMX client as static assets; the Rataui dashboard implements the panel specification of Appendix D against the same broadcast stream, so web and terminal views can never disagree about cycle state.

VI. EVALUATION DESIGN

This section is a pre-registered protocol: *no empirical results are reported in this paper*, and the design states in advance what would confirm or falsify hypothesis H.

A. Benchmarks and Metrics

Table IV lists the public benchmarks—MMLU [63], ARC-Challenge [64], MATH-500 [65], GSM8K [66]—and the custom $\Delta K / \Delta S$ metric that operationalizes H. For a fixed total budget (tokens and dollars), ΔK counts independently audited verified claims held by a system minus those elicitable from the budget-matched union of single models, and the *synergy count* restricts ΔK to items whose provenance derivation spans at least two model families—a direct estimate of $\mu(\Sigma \cap K_T)$ from Theorem 2. ΔS is the analogous count over PAC-certified skill families (11). Theory-grounded telemetry is reported alongside: $\hat{\rho}_t$ (admitted-mass ratio), $\hat{r}_L$ (premise recall), $\hat{\Delta}_R$ (routing advantage via single-arm counterfactual replay), GDI trajectories, rollback counts, and the capability–alignment ratio of [8].

B. Systems Under Test and Baselines

B1 (single models): each M_i alone, zero-shot and chain-of-thought, at the full budget. **B2 (naive ensemble):** majority vote / best-of- n over the same pool at the same total budget [62], [20]. **B3 (routed, memoryless):** Algorithm 3 without the store ($K = S = \emptyset$ throughout)—isolates Δ_R . **B4 (GAUSS-AGENT0, full).** Ablations: NOGRAPH and NOVECTOR (drop one DualRAG path; targets r_L), NOVERIFIER (log-only admission; targets Assumption 1 and should inflate contamination per Proposition 2), NOCRITIC (random curriculum;

TABLE IV
EVALUATION MATRIX. ALL SYSTEMS RUN UNDER IDENTICAL TOTAL TOKEN AND DOLLAR BUDGETS; ACCURACY IS EXACT-MATCH (MATH/GSM8K), STANDARD MULTIPLE-CHOICE SCORING (MMLU/ARC).

Benchmark	Capability	Metric	Role
MMLU [63]	broad knowledge	accuracy	breadth probe
ARC-C [64]	science reasoning	accuracy	reasoning probe
MATH-500 [65]	competition math	exact match	Tier-1-verifiable
GSM8K [66]	arithmetic chains	exact match	Tier-1-verifiable
ΔK / synergy	knowledge growth	audited counts	tests Theorem 2
ΔS	skill growth	certified families	tests Proposition 1

targets β), and UNIFORMROUTER (targets ε_x vs. learned routing). Budget matching is strict: B4’s verification and storage calls are charged against the same ledger as generation.

C. Protocol for Testing H

Five seeds per system; per-benchmark paired bootstrap over items; significance at $p < 0.05$ with Holm correction across benchmarks. *H is supported* if GAUSS-AGENT0 (B4) exceeds both the best single model (B1) and the naive ensemble (B2) on at least three of four benchmarks at matched budget, *and* the audited synergy count is positive with a nonzero lower confidence bound. *H is falsified* (in this instantiation) if B4 fails to beat B2 anywhere, or if the synergy count is statistically zero—in which case Theorem 2 localizes the failure to an empty Σ or a deficient factor in (6), each separately measurable from the ablations. Contamination controls follow [7]: evaluation probes are hashed and the VerifierAgent’s anti-contamination judge rejects any admission whose derivation matches a probe; retrieval is decontaminated against benchmark n -grams; and an independent post-hoc judge audits the final K_T before scoring, assigning the B1 score to any contaminated run.

VII. DISCUSSION

A. Theoretical Limitations

The guarantees are exactly as conditional as stated. Assumption 1 is load-bearing: Tier-2/3 verification is statistical, so the honest reading of Theorem 1 in production is Proposition 2—convergence to the closure with bounded, audit-rate controlled contamination. The closure itself may be small where the composition rule has no purchase (open-ended creative work has few certifiable items), so H should be expected to hold on verification-rich domains first; this mirrors SAHOO’s finding that truthfulness improves less per unit drift than code or math [8]. The information-theoretic statement (10) concerns retrieval-conditioned evidence, not any latent “knowing” inside the models, and strictness is an empirical question, not a theorem. The productivity ρ can be minute— $T(\varepsilon)$ in (8) scales as $1/\rho$ —so the practical content of Lemma 1 is diagnostic: when progress stalls, exactly one of $\beta, \varepsilon_x, r_L, p_g, c_v$ has collapsed, and each is separately instrumented. Finally, weight-frozen RSI inherits the capability ceiling of its pool: PostTrainBench shows weight updates still beat scaffolding on broad post-training [7], so we position GAUSS-AGENT0

as complementary to—not a replacement for—weight-space improvement.

B. Safety Considerations

Knowledge-space RSI relocates rather than removes the classical RSI risks [51], [50]. Goal drift is gated by (17) with SAHOO-calibrated thresholds and checkpointed roll-back; regression risk uses SAHOO’s trend-adjusted estimator as a stopping rule [8]. The verifier is the principal attack surface: a single falsely admitted item can propagate through `derived_from` edges, so falsification triggers a derivation-cascade quarantine (all downstream items revert to candidate), and cross-family quorums reduce—but, given shared pretraining corpora, do not eliminate—correlated error. Reward hacking is a demonstrated behavior of capable agents [7], [49]; our mitigations are verifier-side (provenance checks, anti-contamination judge, Tier-1-first admission) rather than trust-side, and skill execution is sandboxed with no network egress. Drift can also tip slowly across many cycles [27], [28]; the deployment posture therefore follows SAHOO: conservative thresholds early, human review every 10–20 cycles, and circuit breakers in the UI layer (Section IV-H). We do not claim these measures suffice for arbitrarily capable pools; we claim they are *auditable*, which weight-space RSI cannot currently offer at item granularity.

C. Scalability Analysis

Per Proposition 3, cycle cost is linear in batch size and fan-out and dominated by expert calls; retrieval stays $O(\log N)$ (HNSW [45]) plus $O(bL)$ bounded traversal, with SurrealDB’s in-memory HNSW cache sized to the embedding working set [10]. The router is $O(nd^2)$ per decision—negligible. Adding experts grows n linearly and, by Theorem 3, *cannot* reduce asymptotic routed utility (a useless arm is simply never pulled after exploration), so pool growth is monotone in value and linear in cost. Store growth is curbed by consolidation: duplicate claims merge with provenance union, and a task-weighted μ lets eviction target mass the evaluation never touches. The binding scale limit is dollars: with (16) and (8), total spend to tolerance ε is $O(\frac{1}{\rho} \ln \frac{g_0}{\varepsilon} \cdot B \bar{m} c_{\text{llm}})$, which the dashboard forecasts live so operators can trade ε against budget before, not after, the spend.

VIII. CONCLUSION

We presented GAUSS-AGENT0, a framework that relocates recursive self-improvement from model weights to an external, verifiable knowledge-and-skill state composed from frozen frontier LLMs. The contribution is a matched pair: theory that states the weakest conditions we know of under which routed composition strictly dominates its components—a contraction-based convergence guarantee (Theorem 1), a synergy-conditional superadditivity result with an information-theoretic corollary (Theorem 2), and a no-regret routing bound with an explicit routing-advantage term (Theorem 3)—and a system in which every assumption of that theory is a monitored quantity discharged by a named component (Table III),

implemented on a Rust/Tokio stack over SurrealDB with OpenRouter-mediated experts and SAHOO-style drift gating. The evaluation protocol of Section VI is pre-registered and falsification-oriented. If H survives it, weight-frozen RSI offers a path to compounding capability that is auditable item by item and reversible cycle by cycle; if H fails, Lemma 1 says exactly where to look. Either outcome is progress, and that is the standard a framework for self-improving systems should be held to.

ACKNOWLEDGMENTS

The Gaussian Technologies R&D Team thanks all collaborators and reviewers who contributed feedback on this framework. This manuscript was prepared with LLM assistance for drafting and formatting; the team takes full responsibility for all scientific content.

REFERENCES

- [1] J. Schmidhuber, “Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements,” *arXiv preprint cs/0309048*, 2003.
- [2] A. Novikov, N. Vü, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian *et al.*, “AlphaEvolve: A coding agent for scientific and algorithmic discovery,” *arXiv preprint arXiv:2506.13131*, 2025.
- [3] H. Lin, H. Ye, W. Feng, Q. Huang, Y. Li, H. Lim, Z. Li, X. Wang, J. Ma, Y. Liang, and J. Zou, “Can language models discover scaling laws?” in *International Conference on Learning Representations (ICLR)*, 2026.
- [4] P. Xia, K. Zeng, J. Liu, C. Qin, F. Wu, Y. Zhou, C. Xiong, and H. Yao, “Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning,” *arXiv preprint arXiv:2511.16043*, 2025.
- [5] C. Huang, W. Yu, X. Wang, H. Zhang, Z. Li, R. Li, J. Huang, H. Mi, and D. Yu, “R-Zero: Self-evolving reasoning LLM from zero data,” *arXiv preprint arXiv:2508.05004*, 2025.
- [6] A. Zhao, Y. Wu, Y. Yue, T. Wu, Q. Xu, Y. Yue, M. Lin, S. Wang, Q. Wu, Z. Zheng, and G. Huang, “Absolute zero: Reinforced self-play reasoning with zero data,” *arXiv preprint arXiv:2505.03335*, 2025.
- [7] B. Rank, H. Bhatnagar, A. Prabhu, S. Eisenberg, N. Karina, M. Bethge, and M. Andriushchenko, “PostTrainBench: Can LLM agents automate LLM post-training?” *arXiv preprint arXiv:2603.08640*, 2026.
- [8] S. Sahoo, A. Chadha, V. Jain, and D. Chaudhary, “SAHOO: Safe-guarded alignment for high-order optimization objectives in recursive self-improvement,” in *ICLR 2026 Workshop on AI with Recursive Self-Improvement*, 2026, arXiv:2603.06333.
- [9] OpenRouter, “OpenRouter API documentation: A unified interface for LLMs,” <https://openrouter.ai/docs>, 2026.
- [10] SurrealDB Ltd., “SurrealDB documentation: Multi-model database with graph and vector search,” <https://surrealdb.com/docs>, 2026.
- [11] S. Wang, Z. Jiao, Z. Zhang, Y. Peng, X. Ze, B. Yang, W. Wang, H. Wei, and L. Zhang, “Socratic-zero: Bootstrapping reasoning via data-free agent co-evolution,” *arXiv preprint arXiv:2509.24726*, 2025.
- [12] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan, H. Liu, J. Qiu, X. Qi, Y. Wu *et al.*, “A survey of self-evolving agents: On path to artificial super intelligence,” *arXiv preprint arXiv:2507.21046*, 2025.
- [13] B. Liu, L. Guertler, S. Yu, Z. Liu, P. Qi, D. Balcells, M. Liu, C. Tan, W. Shi, M. Lin, W. S. Lee, and N. Jaques, “SPIRAL: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning,” *arXiv preprint arXiv:2506.24119*, 2025.
- [14] Y. Zhai, S. Tao, C. Chen, A. Zou, Z. Chen, Q. Fu, S. Mai, L. Yu, J. Deng, Z. Cao *et al.*, “AgentEvolver: Towards efficient self-evolving agent system,” *arXiv preprint arXiv:2511.10395*, 2025.
- [15] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, “Darwin Gödel machine: Open-ended evolution of self-improving agents,” *arXiv preprint arXiv:2505.22954*, 2025.
- [16] X. Yin, X. Wang, L. Pan, L. Lin, X. Wan, and W. Y. Wang, “Gödel agent: A self-referential agent framework for recursive self-improvement,” *arXiv preprint arXiv:2410.04444*, 2024.

- [17] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai, “Self-taught optimizer (STOP): Recursively self-improving code generation,” *arXiv preprint arXiv:2310.02304*, 2024.
- [18] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kados, and I. Stoica, “RouteLLM: Learning to route LLMs with preference data,” *arXiv preprint arXiv:2406.18665*, 2024.
- [19] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *arXiv preprint arXiv:2305.05176*, 2023.
- [20] D. Jiang, X. Ren, and B. Y. Lin, “LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [21] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, “Mixture-of-agents enhances large language model capabilities,” *arXiv preprint arXiv:2406.04692*, 2024.
- [22] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [23] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [24] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [25] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG: Simple and fast retrieval-augmented generation,” *arXiv preprint arXiv:2410.05779*, 2024.
- [26] X. Wu, S. Yin, Y. Kang, X. Zhang, Q. Xu, Z. Chen, and W. Zhang, “SGM: A statistical Gödel machine for risk-controlled recursive self-modification,” *arXiv preprint arXiv:2510.10232*, 2025.
- [27] S. Han, J. Liu, Y. Su, W. Duan, X. Liu, C. Xie, M. Bansal, M. Ding, L. Zhang, and H. Yao, “Alignment tipping process: How self-evolution pushes LLM agents off the rails,” *arXiv preprint arXiv:2510.04860*, 2025.
- [28] C. Wang, C. Li, S. Liu, Z. Chen, J. Hou, J. Qi, R. Li, L. Zhang, Q. Ye, Z. Liu, X. Chen, X. Zhang, and P. S. Yu, “The devil behind Moltbook: Anthropic safety is always vanishing in self-evolving AI societies,” *arXiv preprint arXiv:2602.09877*, 2026.
- [29] ICLR 2026 Workshop Organizers, “ICLR 2026 workshop on AI with recursive self-improvement,” <https://recursive-workshop.github.io>, 2026.
- [30] J. Qiu, X. Qi, T. Zhang, X. Juan, J. Guo, Y. Lu, Y. Wang, Z. Yao, Q. Ren, X. Jiang *et al.*, “Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution,” *arXiv preprint arXiv:2505.20286*, 2025.
- [31] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, “The AI scientist: Towards fully automated open-ended scientific discovery,” *arXiv preprint arXiv:2408.06292*, 2024.
- [32] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *International Conference on Learning Representations (ICLR)*, 2024.
- [33] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, L. Weng, and A. Mądry, “MLE-bench: Evaluating machine learning agents on machine learning engineering,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [34] H. Wijk, T. Lin, J. Becker, S. Jawhar, N. Parikh, T. Broadley, L. Chan, M. Chen, J. Clymer, J. Dhyani *et al.*, “RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts,” *arXiv preprint arXiv:2411.15114*, 2024.
- [35] G. Starace, O. Jaffe, D. Sherburn, J. Aung, J. S. Chan, L. Maksin, R. Dias, E. Mays, B. Kinsella, W. Thompson *et al.*, “PaperBench: Evaluating AI’s ability to replicate AI research,” *arXiv preprint arXiv:2504.01848*, 2025.
- [36] O. Press, B. Amos, H. Zhao, Y. Wu, S. K. Ainsworth, D. Krupke, P. Kidger *et al.*, “AlgoTune: Can language models speed up general-purpose numerical programs?” *arXiv preprint arXiv:2507.15887*, 2025.
- [37] B. Zhao, D. Magka, M. Jiang, X. Li, R. Raileanu, T. Shavrina, J.-C. Gagnon-Audet, K. Niu *et al.*, “The automated LLM speedrunning benchmark: Reproducing nanoGPT improvements,” *arXiv preprint arXiv:2506.22419*, 2025.
- [38] T. Kwa, B. West, J. Becker, A. Deng, K. Garcia, M. Hasin, S. Jawhar, M. Kinniment, N. Rush, S. Von Arx *et al.*, “Measuring AI ability to complete long tasks,” *arXiv preprint arXiv:2503.14499*, 2025.
- [39] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [40] J. Krajewski, J. Ludziejewski, K. Adamczewski, M. Pióro, M. Krutul, S. Antoniuk, K. Ciebia, K. Król, T. Odrzygóźdź, P. Sankowski *et al.*, “Scaling laws for fine-grained mixture of experts,” *arXiv preprint arXiv:2402.07871*, 2024.
- [41] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, C. Hallacy, J. Leike *et al.*, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [42] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, J. Uesato *et al.*, “Training compute-optimal large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [43] H. Li, W. Zheng, J. Hu, Q. Wang, H. Zhang, Z. Wang, S. Xuyang, Y. Fan, S. Zhou, X. Zhang *et al.*, “Predictable scale: Part I – optimal hyperparameter scaling law in large language model pretraining,” *arXiv preprint*, 2025.
- [44] M. Chen, B. Hui, Z. Cui, J. Yang, D. Liu, J. Sun, J. Lin, and Z. Liu, “Parallel scaling law for language models,” *arXiv preprint arXiv:2505.10475*, 2025.
- [45] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [46] Tokio Contributors, “Tokio: An asynchronous Rust runtime,” <https://tokio.rs>, 2026.
- [47] —, “Axum: Ergonomic and modular web framework for Rust,” <https://github.com/tokio-rs/axum>, 2026.
- [48] Ratatui Contributors, “Ratatui: A Rust library for cooking up terminal user interfaces,” <https://ratatui.rs>, 2026.
- [49] J. Gasteiger, A. Khan, S. Bowman, V. Mikulik, E. Perez, and F. Roger, “Automated researchers can subtly sandbag,” *Anthropic Alignment Science Blog*, 2025.
- [50] V. Krakovna, J. Uesato, V. Mikulik, M. Rahtz, T. Everitt, R. Kumar, Z. Kenton, J. Leike, and S. Legg, “Specification gaming: The flip side of AI ingenuity,” DeepMind Blog, 2020.
- [51] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv preprint arXiv:1606.06565*, 2016.
- [52] A. Tarski, “A lattice-theoretical fixpoint theorem and its applications,” *Pacific Journal of Mathematics*, vol. 5, no. 2, pp. 285–309, 1955.
- [53] S. Banach, “Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales,” *Fundamenta Mathematicae*, vol. 3, pp. 133–181, 1922.
- [54] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006.
- [55] L. G. Valiant, “A theory of the learnable,” *Communications of the ACM*, vol. 27, no. 11, pp. 1134–1142, 1984.
- [56] L. Li, W. Chu, J. Langford, and R. E. Schapire, “A contextual-bandit approach to personalized news article recommendation,” in *Proceedings of the 19th International Conference on World Wide Web (WWW)*, 2010, pp. 661–670.
- [57] W. Chu, L. Li, L. Reyzin, and R. E. Schapire, “Contextual bandits with linear payoff functions,” in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 208–214.
- [58] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge University Press, 2020.
- [59] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.
- [60] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri *et al.*, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [61] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learn-

ing,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- [62] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [63] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2021.
- [64] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafford, “Think you have solved question answering? Try ARC, the AI2 reasoning challenge,” *arXiv preprint arXiv:1803.05457*, 2018.
- [65] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, “Measuring mathematical problem solving with the MATH dataset,” *arXiv preprint arXiv:2103.03874*, 2021.
- [66] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.

APPENDIX A FULL SURREALDB SCHEMA (SURREALQL)

Listing 5 is the complete schema referenced throughout the paper. It targets SurrealDB 2.x [10]: HNSW vector indexes via `DEFINE INDEX ...HNSW`, typed graph edges via `DEFINE TABLE ...TYPE RELATION`, and k -NN search via the `<|k,ef|>` operator.

```

1 DEFINE NAMESPACE gauss; USE NS gauss;
2 DEFINE DATABASE core; USE DB core;
3
4 -- ===== node tables =====
5 DEFINE TABLE concept SCHEMAFULL;
6 DEFINE FIELD name ON concept TYPE string;
7 DEFINE FIELD embedding ON concept TYPE array<float>;
8 DEFINE FIELD created ON concept TYPE datetime VALUE
9   time::now();
10 DEFINE INDEX concept_name ON concept FIELDS name UNIQUE;
11 DEFINE INDEX concept_vec ON concept FIELDS embedding
12   HNSW DIMENSION 1024 DIST COSINE TYPE F32 EFC 200 M 16;
13
14 DEFINE TABLE claim SCHEMAFULL;
15 DEFINE FIELD content ON claim TYPE string;
16 DEFINE FIELD embedding ON claim TYPE array<float>;
17 DEFINE FIELD confidence ON claim TYPE float
18   ASSERT $value >= 0 AND $value <= 1;
19 DEFINE FIELD status ON claim TYPE string DEFAULT 'candidate'
20   ASSERT $value INSIDE ['candidate','verified','falsified'];
21 DEFINE FIELD tier ON claim TYPE int; -- verifier
22   tier 1..3
23 DEFINE FIELD cycle ON claim TYPE int; -- admission
24   cycle t
25 DEFINE FIELD provenance ON claim TYPE object; -- models,
26   premises, judge
27 DEFINE INDEX claim_vec ON claim FIELDS embedding
28   HNSW DIMENSION 1024 DIST COSINE TYPE F32 EFC 200 M 16;
29
30 DEFINE TABLE skill SCHEMAFULL;
31 DEFINE FIELD name ON skill TYPE string;
32 DEFINE FIELD signature ON skill TYPE string;
33 DEFINE FIELD code ON skill TYPE string; -- sandboxed
34   source
35 DEFINE FIELD lang ON skill TYPE string;
36 DEFINE FIELD embedding ON skill TYPE array<float>;
37 DEFINE FIELD pass_rate ON skill TYPE float; -- empirical
38   p_hat
39 DEFINE FIELD ci_low ON skill TYPE float; -- PAC
40   bound, eq. (11)
41 DEFINE FIELD m_tests ON skill TYPE int;
42 DEFINE FIELD cycle ON skill TYPE int;
43 DEFINE INDEX skill_vec ON skill FIELDS embedding
44   HNSW DIMENSION 1024 DIST COSINE TYPE F32 EFC 200 M 16;
45
46 DEFINE TABLE model SCHEMAFULL;
47 DEFINE FIELD slug ON model TYPE string; --
48   OpenRouter slug
49 DEFINE FIELD provider ON model TYPE string;

```

```

42 DEFINE FIELD price_in ON model TYPE float; -- USD / 1e6
43   tokens
44 DEFINE FIELD price_out ON model TYPE float;
45 DEFINE FIELD latency ON model TYPE float; -- rolling
46   p50, ms
47 DEFINE INDEX model_slug ON model FIELDS slug UNIQUE;
48
49 DEFINE TABLE task SCHEMAFULL;
50 DEFINE FIELD family ON task TYPE string;
51 DEFINE FIELD spec ON task TYPE object;
52 DEFINE FIELD probe ON task TYPE bool DEFAULT false; --
53   eval probe?
54
55 DEFINE TABLE snapshot SCHEMAFULL;
56 DEFINE FIELD cycle ON snapshot TYPE int;
57 DEFINE FIELD label ON snapshot TYPE string;
58 DEFINE FIELD taken ON snapshot TYPE datetime VALUE
59   time::now();
60
61 -- ===== relation (edge) tables =====
62 DEFINE TABLE about TYPE RELATION FROM claim TO
63   concept;
64 DEFINE TABLE relates TYPE RELATION FROM concept TO
65   concept;
66 DEFINE TABLE supports TYPE RELATION FROM claim TO
67   claim;
68 DEFINE TABLE contradicts TYPE RELATION FROM claim TO
69   claim;
70 DEFINE TABLE derived_from TYPE RELATION FROM claim TO
71   claim;
72 DEFINE TABLE evidences TYPE RELATION FROM claim TO
73   skill;
74 DEFINE TABLE requires TYPE RELATION FROM skill TO
75   concept;
76 DEFINE TABLE certified_on TYPE RELATION FROM skill TO
77   task;
78 DEFINE TABLE involves TYPE RELATION FROM task TO
79   concept;
80 DEFINE TABLE emitted_by TYPE RELATION FROM claim TO
81   model;
82
83 -- ===== recurring queries =====
84 -- vector path (Alg. 2): k-NN over verified claims
85 SELECT id, content, confidence FROM claim
86 WHERE status = 'verified' AND embedding <|8,40|> $qvec;
87
88 -- graph path seed expansion: claims about linked concepts
89   (2 hops)
90 SELECT <-about<-claim AS premises FROM concept
91 WHERE name INSIDE $entities;
92
93 -- synergy count for Theorem 2(b): multi-family derivations
94 SELECT count() FROM claim
95 WHERE status = 'verified'
96   AND array::len(provenance.model_families) >= 2
97 GROUP ALL;
98
99 -- rollback: revert everything admitted after checkpoint $c
100 UPDATE claim SET status = 'candidate' WHERE cycle > $c;
101 DELETE derived_from WHERE out.cycle > $c OR in.cycle > $c;

```

Listing 5. Complete GAUSS-AGENT0 schema and the recurring queries.

APPENDIX B RUST MODULE INTERFACE SPECIFICATIONS

Listing 6 fixes the cross-crate contracts; everything else is implementation detail behind these traits.

```

1 // ----- core types -----
2 pub struct ModelId(pub String); // OpenRouter
3   slug
4 pub struct Query { pub text: String, pub class: TaskClass,
5   pub embedding: Vec<f32> }
6 pub struct Context { pub premises: Vec<ClaimRef>,
7   pub similar: Vec<ItemRef>,
8   pub packed: String }
9 pub struct Candidate { pub by: ModelId, pub answer: String,
10   pub claims: Vec<DraftClaim>,
11   pub skills: Vec<DraftSkill>,
12   pub usage: Usage }

```

```

12 pub enum Verdict { Pass { tier: u8 }, Fail { reason: String
13 }, Abstain }
14 pub struct Gdi { pub sem: f64, pub lex: f64, pub strc: f64,
15 pub dist: f64, pub total: f64 }
16 // eq. (17)
17 pub struct PacCertificate { pub p_hat: f64, pub ci_low: f64,
18 pub m: usize, pub delta: f64 }
19 // eq. (11)
20 // ----- component contracts -----
21 #[async_trait::async_trait]
22 pub trait Store: Send + Sync {
23     async fn admit(&self, batch: Admitted) -> Result<(),
24     StoreError>;
25     async fn checkpoint(&self, label: String) ->
26     Result<SnapshotId, StoreError>;
27     async fn rollback(&self, to: SnapshotId) -> Result<(),
28     StoreError>;
29     async fn knn(&self, v: Vec<f32>, k: usize, ef: usize)
30     -> Result<Vec<ItemRef>, StoreError>;
31     async fn beam(&self, seeds: Vec<ConceptRef>, b: usize,
32     depth: usize)
33     -> Result<Vec<Path>, StoreError>;
34 }
35 #[async_trait::async_trait]
36 pub trait Router: Send + Sync {
37     fn route(&self, q: &Query) -> Dispatch; //
38     Alg. 3
39     fn reward(&self, arm: ModelId, ctx: Vec<f32>, r: f64);
40     // eq. (4)
41 }
42 pub struct EngineConfig { pub budget: u32, pub eps: f64,
43 pub patience: u32, pub tau_gdi:
44 f64,
45 pub batch: usize }
46 pub struct Engine { /* owns
47 Router+Store+Critic+Verifier+Experts */
48 impl Engine {
49     pub async fn run(self, cfg: EngineConfig) -> FixedPoint
50     { /* Alg. 1 */
51         unimplemented!() }
52 }
53 // ----- event bus consumed by Axum + Ratatui
54 -----
55 pub enum CycleEvent { Started { t: u32 }, Admitted { mass:
56 f64 },
57 Drift { gdi: Gdi }, RolledBack { to:
58 u32 },
59 Converged { t: u32 } }

```

Listing 6. Cross-crate interface specifications (gauss-core).

APPENDIX C OPENROUTER ROUTING CONFIGURATION

Listing 7 is the deployable routing configuration: the expert pool with cost metadata feeding (4), bandit hyperparameters for Algorithm 3, and provider-diverse fallback chains used by Listing 4. Slugs are illustrative and replaceable without code changes.

```

1 {
2   "pool": [
3     { "slug": "openai/gpt-4o", "family":
4     "openai",
5     "roles": ["general", "tool_use"] },
6     { "slug": "anthropic/claude-sonnet-4.5", "family":
7     "anthropic",
8     "roles": ["reasoning", "long_context"] },
9     { "slug": "deepseek/deepseek-r1", "family":
10    "deepseek",
11    "roles": ["math", "code"] },
12    { "slug": "google/gemini-2.5-pro", "family":
13    "google",
14    "roles": ["multimodal", "general"] },
15    { "slug": "meta-llama/llama-3.3-70b-instruct",
16    "family": "meta",

```

TABLE V
TUI PANELS (RATATUI WIDGETS) AND THEIR DATA SOURCES.

Panel	Widget	Source	Quantity shown
Header	Paragraph	engine	$t/B_{\max}$, state, ε , τ
Admission	Sparkline	Admitted	$\mu(A_t)$, $\hat{\rho}_t$, $T(\varepsilon)$ (8)
Drift	GaugeX4	Drift	(17) components vs. τ
Router	Table	router	arm, pulls, $\theta^\top \phi$, ucb, cost
Verifier	BarChart	verifier	queue depth, pass/fail/abstain by tier
Log	List	tracing	rolling structured log tail

```

12     "roles": ["cheap_bulk"] }
13 },
14 "bandit": {
15     "alpha": 0.6, "epsilon_x": 0.05, "gamma": 0.03,
16     "fanout": 3, "context_dim": 768
17 },
18 "costs": { "lambda_dollar": 0.15, "lambda_latency": 0.05
19 },
20 "fallbacks": {
21     "default": ["openai/gpt-4o",
22     "anthropic/claude-sonnet-4.5",
23     "google/gemini-2.5-pro"],
24     "math": ["deepseek/deepseek-r1", "openai/gpt-4o"]
25 },
26 "provider": { "allow_fallbacks": true, "sort":
27     "throughput" },
28 "budget": { "usd_per_cycle": 4.0, "tokens_per_cycle":
29     600000 }

```

Listing 7. Routing configuration (gauss-router/config.json).

APPENDIX D RATATUI TUI COMPONENT SPECIFICATION

The dashboard renders the broadcast stream of CycleEvents at a 250ms tick. Table V specifies the panels; Listing 8 fixes the layout. Keybindings: q quit, p pause/resume, r operator rollback (confirmation modal), t edit thresholds (ε, τ, k), arrow keys navigate the router table.

1	Gauss-Agent0 cycle 12/50 eps 1e-3 GDI 0.31/0.44 [RUN]									
2										
3										
4	Admission mass		Router (LinUCB)		Verifier					
5	rho=0.18 T(eps)=31		arm pulls ucb		T1 #####		91%			
6			gpt4o 412 0.71		T2 #####		63%			
7			sonnet 388 0.69		T3 ###		29%			
8										
9	GDI	sem####	lex###	str####	dist###		0.31			
10										
11	12:04:31	INFO	admitted 47 claims, 3 skills (cycle 12)							
12	12:04:29	WARN	tier-3 abstain rate rising (0.18 -> 0.24)							
13										

Listing 8. Dashboard layout (constraint tree: rows then columns).

APPENDIX E REST API SPECIFICATION (OPENAPI SNIPPET)

Listing 9 is the OpenAPI 3.0 surface served by gauss-server; the WebSocket stream at /v1/events mirrors CycleEvent (Appendix B) as JSON frames.

```

1 openapi: 3.0.3
2 info: { title: Gauss-Agent0 API, version: 0.1.0 }
3 paths:
4   /v1/query:
5     post:
6       summary: Routed, retrieval-grounded answer with
7       attribution
8       requestBody:
9         required: true
10        content:
11          application/json:
12            schema: { $ref: '#/components/schemas/Query' }

```

```

12     responses:
13       '200':
14         description: answer + per-expert weights +
provenance
15         content:
16           application/json:
17             schema: { $ref: '#/components/schemas/Answer'
18         }
19 /v1/rsi/cycles:
20   post: { summary: Start RSI loop with EngineConfig }
21   get: { summary: Status, current t, rho_hat, gdi,
forecast }
22 /v1/rsi/cycles/rollback:
23   post: { summary: Operator rollback to snapshot
(confirmed) }
24 /v1/kg/search:
25   get:
26     summary: Hybrid search over claims and skills
27     parameters:
28       - { name: q, in: query, schema: { type: string } }
29       - { name: k, in: query, schema: { type: integer,

```

```

29     example: 8 } }
30 /v1/kg/subgraph:
31   get: { summary: Typed neighborhood for the web KG view }
32 components:
33   schemas:
34     Query:
35       type: object
36       required: [text]
37       properties:
38         text: { type: string }
39         budget: { type: number, example: 0.02 }
40     Answer:
41       type: object
42       properties:
43         text: { type: string }
44         experts: { type: array, items: { type: object } }
45         sources: { type: array, items: { type: string } }
46         cost_usd: { type: number }

```

Listing 9. OpenAPI snippet (abridged): query, loop control, KG access.